

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**  
**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**  
**KHOA: CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO ĐỒ ÁN THỰC HÀNH**  
**LẬP TRÌNH SOCKET**  
**MÔN MẠNG MÁY TÍNH**

**Thành viên nhóm:**

**Nguyễn Đức Đạt - 24120037**

**Nguyễn Trọng An - 24120017**

**Đỗ Quốc Đại - 24120030**

**Giảng viên lý thuyết: Nguyễn Thanh Quân**

**Giảng viên thực hành: Lê Hà Minh**

**Lớp lý thuyết: 24CTT4**

**Học kỳ - Niên khóa: HK1 - 2025-2026**

# MỤC LỤC

|        |                                                  |    |
|--------|--------------------------------------------------|----|
| 1.     | GIỚI THIỆU ĐỒ ÁN .....                           | 4  |
| 1.1.   | Tổng quan đồ án và phân công công việc .....     | 4  |
| 1.2.   | Phạm vi yêu cầu .....                            | 5  |
| 1.2.1. | Yêu cầu cơ bản .....                             | 5  |
| 1.2.2. | Yêu cầu nâng cao .....                           | 7  |
| 2.     | LÝ THUYẾT NỀN TẢNG .....                         | 8  |
| 2.1.   | Giao thức RTSP .....                             | 8  |
| 2.1.1. | Khái niệm .....                                  | 8  |
| 2.1.2. | Vai trò .....                                    | 8  |
| 2.1.3. | Đặc điểm kỹ thuật .....                          | 8  |
| 2.2.   | Giao thức RTP .....                              | 9  |
| 2.2.1. | Khái niệm .....                                  | 9  |
| 2.2.2. | Vai trò .....                                    | 9  |
| 2.2.3. | Cấu trúc gói RTP .....                           | 9  |
| 2.2.4. | Đặc tính RTP .....                               | 10 |
| 2.3.   | Mối quan hệ giữa RTP và RTSP .....               | 10 |
| 2.4.   | Định dạng MJPEG .....                            | 10 |
| 2.4.1. | Khái niệm .....                                  | 10 |
| 2.4.2. | Đặc điểm .....                                   | 11 |
| 2.4.3. | Định dạng MJPEG trong bài lab .....              | 11 |
| 3.     | THIẾT KẾ HỆ THỐNG .....                          | 12 |
| 3.1.   | Tổng quan hệ thống .....                         | 12 |
| 3.2.   | Thiết kế chi tiết phía Server .....              | 12 |
| 3.2.1. | Các thành phần module .....                      | 12 |
| 3.2.2. | Máy trạng thái .....                             | 13 |
| 3.3.   | Thiết kế chi tiết phía Client .....              | 13 |
| 3.3.1. | Kiến trúc đa luồng .....                         | 13 |
| 3.3.2. | ServerWorker.py (Session Controller) .....       | 13 |
| 3.3.3. | VideoStream.py (Data Access Layer) .....         | 14 |
| 3.3.4. | RtpPacket.py (Encapsulation) .....               | 14 |
| 3.3.5. | Cấu trúc dữ liệu quan trọng .....                | 14 |
| 3.4.   | Thiết kế chi tiết từ phía Server .....           | 15 |
| 4.     | CHI TIẾT THỰC HIỆN .....                         | 17 |
| 4.1.   | Khởi tạo và thiết lập kết nối .....              | 17 |
| 4.1.1. | Khởi tạo Server(Server.py) .....                 | 17 |
| 4.1.2. | Khởi tạo Client (ClientLauncher.py) .....        | 17 |
| 4.1.3. | Lệnh Setup .....                                 | 18 |
| 4.2.   | Truyền tải dữ liệu (RTP/UDP) – Phía Server ..... | 18 |
| 4.2.2. | Đóng gói RTP (RtpPacket.py) .....                | 18 |
| 4.2.3. | Kiểm soát luồng truyền: .....                    | 18 |

|        |                                                |    |
|--------|------------------------------------------------|----|
| 4.3.   | Giai đoạn xử lý và hiển thị - phía Client..... | 19 |
| 4.3.1. | Lắng nghe UDP và chống lỗi.....                | 19 |
| 4.3.2. | Hiển thị và buffering .....                    | 19 |
| 5.     | KẾT QUẢ THỰC NGHIỆM HỆ THỐNG.....              | 20 |
| 5.1.   | Môi trường triển khai (Test Environment).....  | 20 |
| 5.2.   | Các kịch bản kiểm thử .....                    | 20 |
| 5.2.1. | Phát video cơ bản .....                        | 20 |
| 5.2.2  | Nhận xét chung .....                           | 22 |
| 6.     | TÀI LIỆU THAM KHẢO .....                       | 22 |

# 1. GIỚI THIỆU ĐỒ ÁN

## 1.1. Tổng quan đồ án và phân công công việc

Đây là đồ án được đưa ra nhằm củng cố kiến thức của sinh viên trong môn Mạng máy tính bằng việc yêu cầu xây dựng một hệ thống gồm hai thành phần chính: RTSP client và RTP server, hoạt động thông qua hai kênh truyền độc lập. Kênh điều khiển sử dụng giao thức RTSP (Real-Time Streaming Protocol) chạy trên TCP để đảm bảo độ tin cậy của các lệnh điều khiển, trong khi kênh truyền dữ liệu video sử dụng RTP (Real-Time Transport Protocol) chạy trên UDP nhằm đạt tốc độ truyền cao và độ trễ thấp, phù hợp với yêu cầu video thời gian thực.

### **Bảng phân chia công việc và mức độ hoàn thành:**

| Nhiệm vụ                                | Họ và tên       | Mức độ hoàn thành |
|-----------------------------------------|-----------------|-------------------|
| Cài đặt RTSP protocol ở phía Client     | Nguyễn Đức Đạt  | Xuất sắc, 100%    |
| Cài đặt RTP packetization ở phía Server | Nguyễn Đức Đạt  | Xuất sắc, 100%    |
| HD Video Streaming                      | Nguyễn Trọng An | Xuất sắc, 100%    |
| Client-Side Caching                     | Đỗ Quốc Đại     | Xuất sắc, 100%    |
| Viết báo cáo                            | Đỗ Quốc Đại     | Xuất sắc, 100%    |

## 1.2. Phạm vi yêu cầu

### 1.2.1. Yêu cầu cơ bản

Là các yêu cầu bắt buộc để hệ thống có thể hoạt động đúng chức năng theo chuẩn RTSP/RTP.

#### + **RTSP CLIENT**

Sinh viên cần hoàn thiện phần xử lý RTSP trong Client:

##### - **SETUP**

- Gửi yêu cầu thiết lập session.
- Gửi Transport header khai báo cổng RTP.
- Đọc và lưu Session ID.
- Tạo socket UDP để nhận RTP.

##### - **PLAY**

- Gửi yêu cầu bắt đầu stream.
- Chèn Session header (dùng Session ID).

##### - **PAUSE**

- Gửi yêu cầu tạm dừng.
- Chèn Session header.

##### - **TEARDOWN**

- Gửi yêu cầu kết thúc session.
- Chèn Session header.

- **Các yêu cầu kỹ thuật khác:**

- Gửi request theo đúng format RTSP/1.0.
- Quản lý thứ tự request bằng trường CSeq (tăng dần).
- Quản lý state machine của client: INIT → READY → PLAYING → READY → INIT.

+ **RTP PACKETIZATION(SERVER)**

Sinh viên cần hoàn thiện hàm encode() trong class RtpPacket:

- **Gói tin RTP phải tuân theo chuẩn**

- Version (V = 2).
- Padding (P = 0), Extension (X = 0), CC = 0.
- Marker (M = 0).
- Payload type (PT = 26 — MJPEG).
- Sequence number (từ frame number).
- Timestamp (dùng time.time()).
- SSRC (chọn số bất kỳ).
- Sao chép payload (JPEG frame) đúng định dạng.

- **Server phải**

- Đọc từng frame từ VideoStream.
- Đóng gói frame → RTP packet.
- Gửi packet qua UDP mỗi 50 ms

### 1.2.2. Yêu cầu nâng cao

Là các yêu cầu mở rộng vào để hệ thống có thể hoạt động trôi chảy, mượt mà và đạt năng suất cao hơn, cụ thể:

#### + **HD VIDEO STREAMING**

- Hệ thống phải hỗ trợ streaming video chất lượng cao (720p/1080p):
  - **Xử lý MJPEG** có kích thước lớn ( $> 97\text{KB}$ ).
  - **Fragmentation:** tách 1 frame thành nhiều RTP packet nhỏ (Payload  $\leq 1400$  bytes) để không vượt quá MTU của mạng.
  - **Flow Control (Burst Mode):** Server điều tiết tốc độ gửi gói tin theo chùm để cân bằng giữa tốc độ nạp bộ đệm và khả năng chịu tải của mạng, giảm thiểu mất gói.

#### + **CLIENT-SIDE CACHING**

- Hệ thống phải có bộ đệm để chống giật hình và cải thiện mượt mà:
  - **Smart Buffering:** Sử dụng hàng đợi (Queue) với cơ chế ngưỡng (Threshold). Video chỉ bắt đầu phát khi bộ đệm nạp đủ dữ liệu, mô phỏng trải nghiệm mượt mà. Khi người dùng nhấn PLAY, video phát từ buffer chứ không phải từ socket trực tiếp.
  - **Memory Caching:** Lưu trữ tạm thời các khung hình đã hiển thị vào RAM. Cho phép người dùng xem lại (Instant Replay) các đoạn video đã qua mà không tiêu tốn băng thông mạng và không có độ trễ.

## **2. LÝ THUYẾT NỀN TẢNG**

### **2.1. Giao thức RTSP**

#### **2.1.1. Khái niệm**

RTSP (RFC 2326) là một giao thức điều khiển tầng ứng dụng (Application Layer), được thiết kế để thiết lập và kiểm soát các phiên truyền thông tin thời gian thực (như âm thanh, video). Khác với HTTP là giao thức phi trạng thái (stateless), RTSP là giao thức có trạng thái (stateful), nghĩa là máy chủ lưu giữ thông tin về phiên làm việc của client từ lúc khởi tạo cho đến khi kết thúc.

#### **2.1.2. Vai trò**

- + RTSP không truyền video trực tiếp mà chỉ dùng để gửi lệnh điều khiển, ví dụ:
  - SETUP – thiết lập phiên và thông số truyền tải
  - PLAY – bắt đầu phát
  - PAUSE – tạm dừng
  - TEARDOWN – kết thúc phiên
- + Media thực tế được truyền bằng RTP.

#### **2.1.3. Đặc điểm kỹ thuật**

- + Giao thức dạng text, giống HTTP.
- + Thường chạy qua TCP (đảm bảo tin cậy cho lệnh điều khiển).
- + Cổng mặc định: 554 TCP.
- + Hỗ trợ nhiều session đồng thời nhờ Session ID.
- + Có bộ đếm CSeq để đánh số thứ tự các yêu cầu.



## **2.2. Giao thức RTP**

### **2.2.1. Khái niệm**

RTP (Real-Time Transport Protocol) là giao thức tầng ứng dụng cho phép truyền tải dữ liệu media thời gian thực qua mạng IP. Được thiết kế để truyền video, audio với độ trễ thấp.

### **2.2.2. Vai trò**

- + Trong hệ thống streaming:
  - RTSP gửi lệnh điều khiển.
  - RTP vận chuyển dữ liệu media.
- + RTP thường được truyền qua UDP để giảm độ trễ.

### **2.2.3. Cấu trúc gói RTP**

- + Gói RTP gồm hai phần:
  - Header (12 bytes, chứa phiên bản, sequence number, timestamp, SSRC)
  - Payload (dữ liệu video/audio)
- + Các trường quan trọng:
  - Version (V): phải là 2.
  - Payload Type (PT): xác định loại dữ liệu — MJPEG là 26.
  - Sequence Number: đánh số gói → hỗ trợ phát hiện mất gói.
  - Timestamp: hỗ trợ đồng bộ khung hình.
  - SSRC: định danh nguồn phát.

#### 2.2.4. Đặc tính RTP

Truyền theo kiểu best-effort (UDP) → có thể mất gói.

Không đảm bảo thứ tự → client dựa vào sequence number để sắp xếp.

Là nền tảng của hầu hết các hệ thống live streaming (WebRTC, VoIP...).

### 2.3. Mối quan hệ giữa RTP và RTSP

RTSP đóng vai trò điều khiển phiên, RTP mang dữ liệu. Quy

trình diễn hình:

- Client gửi SETUP → thương lượng cổng RTP.
- Server mở luồng RTP.
- Client gửi PLAY → media bắt đầu truyền qua RTP/UDP.
- Client muốn dừng → gửi PAUSE.
- Kết thúc → TEARDOWN.

Như vậy, RTSP là “remote control”, còn RTP là “đường truyền video”.

### 2.4. Định dạng MJPEG

#### 2.4.1. Khái niệm

MJPEG là định dạng video trong đó:

- Mỗi frame là một ảnh JPEG riêng biệt
- Các ảnh được ghép liên tục tạo thành chuỗi video.

- Không có nén liên frame (no inter-frame compression) như H.264 → dễ xử lý, giải mã nhanh, thích hợp cho lab.

#### **2.4.2. Đặc điểm**

- + Mỗi khung hình độc lập → dễ truyền qua RTP.
- + Giải mã nhẹ → phù hợp lab/streaming độ phân giải thấp.
- + Nhược điểm: file lớn hơn nhiều so với H.264.

#### **2.4.3. Định dạng MJPEG trong bài lab**

- + Video được lưu theo cấu trúc:

[5-byte size][JPEG image]

[5-byte size][JPEG image]

...

- + Trong đó 5 byte đầu chứa kích thước ảnh (số bit).
- + Server đọc từng frame → gói vào RTP → gửi cho client.

### **3. THIẾT KẾ HỆ THỐNG**

#### **3.1. Tổng quan hệ thống**

- + Hệ thống được thiết kế theo mô hình Client-Server với kiến trúc xử lý song song (Multi-threading) và điều khiển ngoài băng tần (Out-of-band Control).
- + Hệ thống bao gồm hai kênh giao tiếp hoạt động độc lập:
  - Kênh điều khiển (Control Channel): Sử dụng giao thức RTSP trên nền TCP Port X. Đảm bảo tính tin cậy tuyệt đối cho các lệnh điều hướng (SETUP, PLAY, PAUSE, TEARDOWN).
  - Kênh dữ liệu (Data Channel): Sử dụng giao thức RTP trên nền UDP (Port Y). Tối ưu hóa tốc độ truyền tải và độ trễ thấp (Low Latency) cho luồng video thời gian thực.

#### **3.2. Thiết kế chi tiết phía Server**

##### **3.2.1. Các thành phần module**

- + Server.py(Connection Listener): Đóng vai trò cổng tiếp đón, sử dụng vòng lặp vô tận để lắng nghe yêu cầu kết nối TCP. Với mỗi Client kết nối, Server khởi tạo một luồng xử lý riêng biệt (ServerWorker), cho phép phục vụ nhiều người dùng đồng thời.
- + ServerWorker.py (Session Controller): Quản lý máy trạng thái (State Machine) của phiên RTSP. Điều tiết tốc độ gửi dữ liệu RTP bằng cơ chế gửi tuần tự theo chu kỳ thời gian cố định, kết hợp chia nhỏ frame để phù hợp với MTU.
- + VideoStream.py (Data Access Layer): Chịu trách nhiệm đọc và truy xuất dữ liệu từ file vật lý. Module đọc toàn bộ file video vào bộ nhớ, tách các khung hình JPEG và lưu vào danh sách, cho phép truy xuất nhanh từng frame theo thứ tự.
- + RtpPacket.py (Encapsulation): Đóng gói dữ liệu thô vào cấu trúc gói tin RTP chuẩn (RFC 1889). Thực hiện đóng gói payload vào gói RTP theo chuẩn RFC 1889, thiết

lập các trường header như version, sequence number, timestamp, payload type và marker bit.

### 3.2.2. Máy trạng thái

- + Server hoạt động theo mô hình máy trạng thái với ba trạng thái chính:
  - **INIT:** Chưa thiết lập phiên RTSP, chưa mở file video và chưa cấp phát tài nguyên truyền tải.
  - **READY:** Phiên RTSP đã được khởi tạo (có Session ID), file video đã được mở, server sẵn sàng phục vụ yêu cầu phát.
  - **PLAYING:** Server đang truyền dữ liệu video cho client bằng các gói RTP thông qua giao thức UDP.

## 3.3. Thiết kế chi tiết phía Client

Client được thiết kế theo kiến trúc đa luồng nhằm đảm bảo giao diện mượt và truyền dữ liệu liên tục.

### 3.3.1. Kiến trúc đa luồng

- + Client sử dụng 2 luồng chính hoạt động song song:
  - **Luồng giao diện:** Chịu trách nhiệm xử lý các sự kiện người dùng (nhấn nút điều khiển) và hiển thị các khung hình video lên màn hình thông qua thư viện Tkinter.
  - **Luồng mạng:** Thực hiện lắng nghe dữ liệu RTP qua UDP, lắp ráp các gói RTP thành khung hình hoàn chỉnh, phát hiện mất gói tin và đưa các khung hình hợp lệ vào bộ đệm để phát lại mượt mà.

### 3.3.2. ServerWorker.py (Session Controller)

Đóng vai trò bộ điều khiển phiên RTSP (Session Controller), chịu trách nhiệm quản lý máy trạng thái của phiên làm việc. Ngoài ra, module này điều khiển quá trình gửi dữ liệu video qua RTP bằng cách phát các gói tin theo chu kỳ thời gian cố định và thực hiện chia

nhỏ dữ liệu khung hình để phù hợp với kích thước MTU của mạng.

### 3.3.3. VideoStream.py (Data Access Layer)

Chịu trách nhiệm đọc dữ liệu video từ file vật lý. Khi khởi tạo, module đọc toàn bộ file MJPEG vào bộ nhớ, tách các khung hình JPEG dựa trên marker bắt đầu và kết thúc, và lưu trữ chúng trong một cấu trúc danh sách để phục vụ việc truy xuất tuần tự từng frame trong quá trình streaming.

### 3.3.4. RtpPacket.py (Encapsulation)

Module RtpPacket.py đảm nhiệm chức năng **đóng gói (encapsulation)** dữ liệu payload vào cấu trúc **gói tin RTP** theo chuẩn RTP (RFC 1889). Module này chịu trách nhiệm xây dựng và phân tích header RTP, bao gồm các trường như version, sequence number, timestamp, payload type và marker bit.

Việc **chia nhỏ dữ liệu khung hình (fragmentation)** để phù hợp với kích thước MTU **không được thực hiện trong module này**, mà được xử lý ở tầng cao hơn trong ServerWorker.py. RtpPacket.py chỉ tập trung vào việc tạo và giải mã các gói RTP từ payload đã được chuẩn bị sẵn.

.

### 3.3.5. Cấu trúc dữ liệu quan trọng

+ **bytearray – Bộ đệm ghép frame**: Được sử dụng để ghép payload của nhiều gói RTP liên tiếp thành một khung hình JPEG hoàn chỉnh trước khi tiến hành giải mã.

- Vai trò:**
- Lưu trữ dữ liệu nhị phân của khung hình.
  - Hỗ trợ thao tác nối và xóa dữ liệu nhanh.
  - Làm bộ đệm tạm thời trong quá trình ghép frame từ nhiều gói RTP.

+ **deque – Bộ đệm frame phát video:** Được sử dụng như một hàng đợi FIFO để lưu trữ các frame đã được decode thành công, phục vụ cho quá trình phát video liên tục và ổn định.

- Vai trò:**
- Lưu trữ tạm thời các frame đã giải mã.
  - Cho phép đệm trước một số frame để giảm ảnh hưởng của jitter mạng.
  - Hỗ trợ phát video mượt hơn.

+ **Event – Cơ chế đồng bộ luồng:** Được sử dụng để điều khiển và đồng bộ hoạt động của các luồng trong hệ thống, đặc biệt là việc **tạm dừng hoặc kết thúc luồng gửi/nhận dữ liệu** khi xảy ra các sự kiện **PAUSE/TEARDOWN**.

+ **Các biến thông kê và điều khiển:**

- **frameNbr:** đếm số khung hình được giải mã thành công
- **totalLostFrames:** thông kê số khung hình bị lỗi (không decode được)

### 3.4. Thiết kế chi tiết từ phía Server

Quy trình từ lúc dữ liệu nằm trên đĩa cứng Server đến khi hiển thị trên màn hình Client trải qua các bước:

#### **Bước 1: Client gửi yêu cầu SETUP (RTSP)**

- Client gửi yêu cầu SETUP đến Server thông qua kết nối TCP (RTSP).
- Server nhận yêu cầu, khởi tạo một phiên làm việc mới (session), tạo đối tượng VideoStream để chuẩn bị dữ liệu video.
- Server trích xuất cổng RTP mà Client cung cấp để dùng cho việc truyền dữ liệu video sau này.
- Trạng thái phiên chuyển từ **INIT** → **READY**.

#### **Bước 2: Server đọc dữ liệu video từ đĩa**

Khi đối tượng VideoStream được khởi tạo, Server:

- Mở file video MJPEG trên đĩa cứng.
- Đọc toàn bộ nội dung file vào bộ nhớ.
- Tách các khung hình JPEG dựa trên marker bắt đầu (FFD8) và kết thúc (FFD9).

- Lưu các frame vào danh sách nội bộ để truy xuất tuần tự.

### **Bước 3: Client gửi yêu cầu PLAY**

- Client gửi yêu cầu PLAY thông qua RTSP.
- Server chuyển trạng thái từ READY → PLAYING.
- Server tạo socket UDP và khởi động một luồng riêng để gửi dữ liệu RTP định kỳ.

### **Bước 4: Server lấy frame video và chia nhỏ dữ liệu**

Trong luồng gửi RTP:

- Server gọi nextFrame() để lấy từng frame JPEG theo thứ tự.
- Mỗi frame được chia nhỏ thành nhiều đoạn dữ liệu (payload) sao cho kích thước không vượt quá MTU mạng.
- Việc chia nhỏ được thực hiện tại Server để đảm bảo mỗi gói RTP có thể truyền an toàn qua mạng.

### **Bước 5: Đóng gói dữ liệu vào gói RTP**

- Mỗi payload được đóng gói thành một gói RTP thông qua lớp RtpPacket.
- Header RTP được thiết lập bao gồm:
  - Version
  - Payload Type (MJPEG)
  - Sequence Number
  - Timestamp
  - Marker bit (đánh dấu gói cuối của một frame)
- Marker bit được đặt bằng 1 cho gói cuối cùng của mỗi frame để Client xác định ranh giới khung hình.

### **Bước 6: Truyền gói RTP qua mạng**

- Các gói RTP được gửi từ Server đến Client thông qua UDP socket.
- Dữ liệu được truyền theo thứ tự các payload đã được chia nhỏ.
- Quá trình này được lặp lại theo chu kỳ thời gian cố định để đảm bảo tốc độ phát video ổn định.



### **Bước 7: Client nhận, ghép và giải mã frame**

- Client nhận các gói RTP, giải mã header và trích xuất payload.
- Payload của nhiều gói RTP được ghép lại bằng bộ đệm bytearray.
- Khi nhận được gói có marker bit = 1:
  - Client tiến hành decode dữ liệu JPEG thành ảnh.
  - Frame hợp lệ được đưa vào hàng đợi deque để chuẩn bị phát.

### **Bước 8: Hiển thị video trên màn hình Client**

- Client lấy frame từ bộ đệm deque theo cơ chế FIFO.
- Frame được hiển thị lên giao diện đồ họa theo tốc độ khung hình đã định.
- Việc đệm frame giúp giảm ảnh hưởng của jitter mạng và đảm bảo video phát mượt hơn.

## **4. CHI TIẾT THỰC HIỆN**

### **4.1. Khởi tạo và thiết lập kết nối**

Giai đoạn này thiết lập kênh điều khiển giữa Server với Client.

#### **4.1.1. Khởi tạo Server(Server.py)**

- Khi chạy file Server.py, server tạo một socket TCP để lắng nghe các kết nối RTSP từ client. Server bind socket với địa chỉ IP và cổng xác định, sau đó chuyển sang trạng thái chờ kết nối.
- Khi có client kết nối đến, server chấp nhận kết nối TCP và khởi tạo một đối tượng ServerWorker để quản lý phiên RTSP tương ứng. Mỗi ServerWorker chịu trách nhiệm xử lý toàn bộ quá trình điều khiển phiên, bao gồm các yêu cầu RTSP như SETUP, PLAY, PAUSE và TEARDOWN.
- Trong giai đoạn SETUP, server khởi tạo đối tượng VideoStream để chuẩn bị dữ liệu video, lưu thông tin cổng RTP của client và sinh mã định danh phiên (Session ID). Việc tạo socket UDP để gửi dữ liệu RTP chỉ được thực hiện khi server nhận yêu cầu PLAY từ client.

#### **4.1.2. Khởi tạo Client (ClientLauncher.py)**

- Khi chạy chương trình Client.py, client đọc các tham số cấu hình từ dòng lệnh, bao gồm địa chỉ IP của Server, cổng RTSP, cổng RTP và tên file video cần phát.
- Sau khi đọc các tham số này, client khởi tạo cửa sổ giao diện đồ họa bằng thư viện

Tkinter và tạo một đối tượng của lớp Client. Đối tượng này chịu trách nhiệm thiết lập kết nối RTSP với Server, gửi các yêu cầu điều khiển và xử lý dữ liệu video nhận được trong suốt phiên streaming.

#### 4.1.3. Lệnh Setup

- Client gửi lệnh RTSP SETUP đến Server để yêu cầu thiết lập phiên streaming. Lệnh này bao gồm tên file video cần phát và cổng RTP mà Client sử dụng để nhận dữ liệu video.
- Tại Server, hàm processRtspRequest() trong module ServerWorker.py tiếp nhận và xử lý lệnh SETUP. Server khởi tạo đối tượng VideoStream để chuẩn bị dữ liệu video, sinh mã định danh phiên (Session ID), lưu thông tin cổng RTP của Client và phản hồi lại Client với mã trạng thái **200 OK**.
- Sau khi lệnh SETUP được xử lý thành công, phiên RTSP được thiết lập và trạng thái của cả Client và Server chuyển sang **READY**, sẵn sàng cho quá trình truyền dữ liệu video khi nhận lệnh PLAY.

### 4.2. Truyền tải dữ liệu (RTP/UDP) – Phía Server

- Quá trình truyền tải dữ liệu video từ Server đến Client được thực hiện thông qua giao thức RTP trên nền UDP. Các hoạt động đóng gói và gửi dữ liệu được xử lý trong module **ServerWorker.py**.

#### 4.2.2. Đóng gói RTP (RtpPacket.py)

- Mỗi mảnh dữ liệu payload sau khi được chia nhỏ từ một khung hình JPEG sẽ được đóng gói vào một gói RTP có header dài 12 byte. Header RTP bao gồm các trường cơ bản như version, payload type, sequence number, timestamp và marker bit.
- Bit marker được đặt bằng 1 đối với gói RTP cuối cùng của mỗi khung hình, cho phép Client xác định ranh giới giữa các frame để tiến hành ghép và giải mã dữ liệu ảnh.

#### 4.2.3. Kiểm soát luồng truyền:

- Server kiểm soát tốc độ gửi dữ liệu RTP bằng cách chèn một khoảng trễ cố định giữa các lần gửi khung hình thông qua cơ chế `event.wait(0.05)` trong luồng gửi RTP.
- Mỗi khung hình JPEG được chia thành nhiều payload có kích thước tối đa khoảng 1460 byte, phù hợp với MTU của mạng Ethernet. Các payload của cùng một khung hình được gửi liên tiếp; gói RTP cuối cùng được đánh dấu bằng marker bit để Client nhận biết thời điểm kết thúc khung hình.

### **4.3. Giai đoạn xử lý và hiển thị - phía Client**

Các hoạt động xử lý dữ liệu nhận được và hiển thị video cho người dùng được thực hiện trong module `Client.py`, với mục tiêu đảm bảo phát video ổn định và mượt mà..

#### **4.3.1. Lắng nghe UDP và chống lỗi**

- Hàm `listenRtp()` chịu trách nhiệm lắng nghe và nhận các gói RTP thông qua socket UDP. Client giải mã header RTP để trích xuất payload và ghép payload của nhiều gói liên tiếp vào một bộ đệm tạm thời.
- Client sử dụng bit marker trong header RTP để xác định gói cuối cùng của một khung hình. Khi nhận đủ dữ liệu của một frame, client tiến hành giải mã dữ liệu JPEG. Nếu quá trình giải mã thất bại, frame đó được xem là lỗi và bị loại bỏ.
- Ngoài ra, client thu thập các thông số thống kê như bitrate và số lượng frame bị lỗi (`totalLostFrames`) nhằm đánh giá chất lượng truyền dữ liệu.

#### **4.3.2. Hiển thị và buffering**

Client sử dụng bộ đệm `frameBuffer` (kiểu `deque`) để lưu trữ các khung hình đã được giải mã thành công trước khi hiển thị. Hàm `bufferPlayer()` chỉ bắt đầu phát video khi số lượng frame trong bộ đệm đạt đến một ngưỡng tối thiểu, giúp giảm hiện tượng giật và ảnh hưởng của jitter mạng.

Các frame sau đó được hiển thị tuần tự thông qua hàm `updateMovie()` với tốc độ khung hình cố định, đảm bảo trải nghiệm xem video ổn định cho người dùng.

## 5. KẾT QUẢ THỬ NGHIỆM HỆ THỐNG

Sau khi hoàn thiện phần cài đặt Server và Client theo mô hình RTSP/RTP, nhóm đã tiến hành chạy thử nghiệm trên máy cá nhân theo cấu hình mạng LAN nội bộ. Các thử nghiệm tập trung vào việc đánh giá mức độ ổn định của phiên RTSP, khả năng truyền frame qua RTP cũng như độ trễ khi phát video.

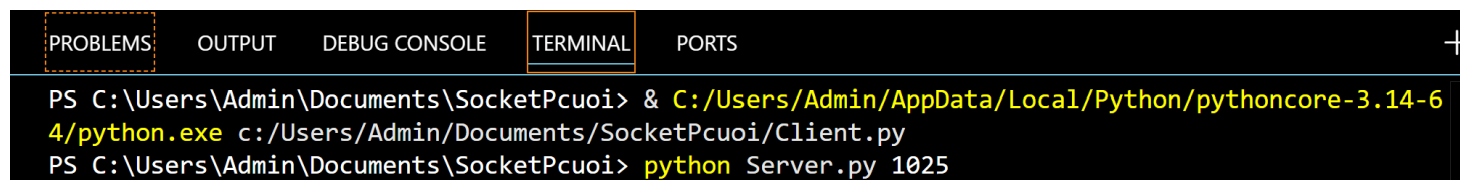
### 5.1. Môi trường triển khai (Test Environment)

- + Thông tin phần cứng: Intel Core Ultra 5 125H, Ram: 32 GB
- + Phần mềm sử dụng: Visual studio code
- + Thư viện sử dụng: Pillow, Tkinter
- + Dữ liệu thử nghiệm: video đầu vào: sampleHD.mjpeg

### 5.2. Các kịch bản kiểm thử

#### 5.2.1. Phát video cơ bản

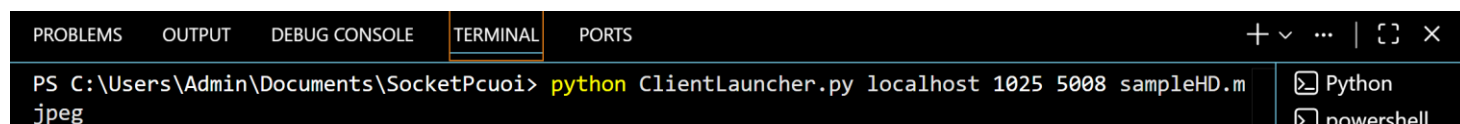
**Terminal 1: python Server.py 1025**



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The command prompt is 'PS C:\Users\Admin\Documents\SocketPcuoi>'. The first command entered is '& C:/Users/Admin/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Admin/Documents/SocketPcuoi/Client.py', which is highlighted in yellow. The second command entered is 'python Server.py 1025'.

```
PS C:\Users\Admin\Documents\SocketPcuoi> & C:/Users/Admin/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Admin/Documents/SocketPcuoi/Client.py
PS C:\Users\Admin\Documents\SocketPcuoi> python Server.py 1025
```

**Terminal 2: python ClientLauncher.py localhost 1025 5008 sampleHD.mjpeg**



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The command prompt is 'PS C:\Users\Admin\Documents\SocketPcuoi>'. The command entered is 'python ClientLauncher.py localhost 1025 5008 sampleHD.mjpeg', which is highlighted in yellow. The terminal window has a title bar that says 'Python' and a button that says 'powershell'.

```
PS C:\Users\Admin\Documents\SocketPcuoi> python ClientLauncher.py localhost 1025 5008 sampleHD.mjpeg
```

**SETUP:**

15  
16  
17  
18  
19  
20

READY = 1  
PLAYING = 2  
state = INIT  
  
SETUP = 0  
PLAY = 1

PROBLEMS
OUTPUT
DEBUG CONSOLE
**TERMINAL**

PS C:\Users\Admin\Documents\SocketPcuoi> jpeg  
  
Data sent:  
SETUP sampleHD.mjpeg RTSP/1.0  
CSeq: 1  
Transport: RTP/UDP; client\_port= 5008  
  
BUFFERING...

RTPClient

Setup

Play

Pause

Teardown

**PLAYING:**

RTPClient



Setup

**Play**

Pause

Teardown

movie.Mjpeg  
RtpPacket.py  
sampleHD.mjpeg  
Server.py  
ServerWorker.py  
VideoStream.py

Current RTP Frame Seq Num: 95  
Current RTP Frame Seq Num: 96  
Current RTP Frame Seq Num: 97  
Current RTP Frame Seq Num: 98  
Current RTP Frame Seq Num: 99  
Current RTP Frame Seq Num: 100  
Current RTP Frame Seq Num: 101  
Current RTP Frame Seq Num: 102  
Current RTP Frame Seq Num: 103  
Current RTP Frame Seq Num: 104  
Current RTP Frame Seq Num: 105  
Bitrate: 7648.50 kbps, Total Lost Frames: 0, Loss Rate: 0.00%  
Current RTP Frame Seq Num: 106  
Current RTP Frame Seq Num: 107  
Current RTP Frame Seq Num: 108  
Current RTP Frame Seq Num: 109  
Current RTP Frame Seq Num: 110

OUTLINE

TIMELINE

### 5.2.2 Nhận xét chung

- Hệ thống đã đáp ứng được các yêu cầu cơ bản của một ứng dụng truyền phát video theo thời gian thực thông qua giao thức RTP/UDP trong môi trường mạng nội bộ.

## 6. TÀI LIỆU THAM KHẢO

- Lập trình Socket cơ bản: <https://realpython.com/python-sockets/>
- Đa luồng trong Python: <https://stackoverflow.com/questions/68425239/how-to-handle-multithreading-with-sockets-in-python>
- Lập trình GUI bằng Python, sử dụng Tkinter:  
<https://www.youtube.com/watch?v=m9kzSDlzpLE&list=PLBfgLd7V3wpXXBLgiQExUPe hRxmAWk2XY>
- Chat GPT